

Aula 13 - Definição do Projeto Final (Parte Prática)

Duração: 2 horas

Objetivos:

- Definir a arquitetura técnica do projeto
- Implementar um MVP funcional (Java + Spring Boot)
- Configurar métricas iniciais de qualidade

Parte Teórica (30 min)

1. Escolha da Arquitetura

Opção	Prós	Contras
Monolito	Simples de desenvolver	Menos escalável
Microserviços	Escalabilidade	Complexidade operacional

2. Tech Stack Básica

- **Backend:** Spring Boot 3.x + Spring Data JPA
- **Testes:** JUnit 5, Mockito, TestContainers
- **Qualidade:** SonarQube + JaCoCo
- **Documentação:** Swagger + Markdown

3. Critérios de MVP

- ✓ 1 Entidade CRUD funcional
- ✓ 1 Teste de integração
- ✓ Pipeline CI básico

Atividade Prática (1h30min)

Passo 1: Setup Inicial

1. Crie o projeto no [Spring Initializr](#):

- Dependências: Spring Web , Spring Data JPA , Lombok , H2 Database

2. Clone o repositório:

bash

```
git clone <seu-repositorio>
cd <projeto>
```

Passo 2: Implementação da Primeira Entidade

Exemplo: CRUD de Produtos

1. Modelo (JPA Entity)

java

```
// src/main/java/com/example/projeto/domain/model/Produto.java
@Entity
@Data // Lombok
@NoArgsConstructor
public class Produto {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String nome;

    @Column(precision = 10, scale = 2)
    private BigDecimal preco;
}
```

2. Repository

java

```
// src/main/java/com/example/projeto/domain/repository/ProdutoRepository.java
public interface ProdutoRepository extends JpaRepository<Produto, Long> {
}
```

3. Controller (REST API)

java

```
// src/main/java/com/example/projeto/interface/rest/ProdutoController.java
@RestController
```

```

@RequestMapping("/produtos")
@RequiredArgsConstructor // Lombok
public class ProdutoController {
    private final ProdutoRepository repository;

    @GetMapping
    public List<Produto> listar() {
        return repository.findAll();
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Produto criar(@RequestBody Produto produto) {
        return repository.save(produto);
    }
}

```

Passo 3: Testes Automatizados

1. Teste de Integração

```

java

// src/test/java/com/example/projeto/interface/rest/ProdutoControllerIT.java
@SpringBootTest
@AutoConfigureMockMvc
class ProdutoControllerIT {
    @Autowired
    MockMvc mockMvc;

    @Test
    void deveListarProdutosVazios() throws Exception {
        mockMvc.perform(get("/produtos"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$", hasSize(0)));
    }
}

```

2. Teste Unitário com Mock

```

java

// src/test/java/com/example/projeto/interface/rest/ProdutoControllerTest.java
@ExtendWith(MockitoExtension.class)
class ProdutoControllerTest {
    @Mock
    ProdutoRepository repository;
}

```

```
@InjectMocks
ProdutoController controller;

@Test
void deveChamarRepositoryAoListar() {
    controller.listar();
    verify(repository).findAll();
}
}
```

Passo 4: Configuração de Qualidade

1. Adicione ao pom.xml :

```
xml

<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.10</version>
</plugin>
```

2. Execute e verifique a cobertura:

```
bash

mvn clean test jacoco:report
# Relatório em: target/site/jacoco/index.html
```

Checklist de Entrega

- CRUD funcional de 1 entidade
- 1 teste de integração e 1 unitário
- Relatório JaCoCo gerado
- Push no repositório com tag v0.1

```
bash

git tag -a v0.1 -m "MVP inicial"
git push origin v0.1
```

Próximos Passos

- **Aula 14:** Definição das user stories completas
- **Aula 15:** Implementação da Sprint 1

Dúvidas?

markdown

> "Como decidir entre H2 e PostgreSQL para testes?"

> ****Resposta:**** Use H2 para desenvolvimento inicial e TestContainers com PostgreSQL em CI

Recursos:

- [Spring Data JPA Docs](#)
- [Jacoco Exemplos](#)

Pronto para codar seu MVP? 🚀 💻